DAY 02

# Linux Command-Line Hands-on Session

Team Kernel Panic · Team-02

| | |
|---|---|
| **Session focus** | Grep, Cut, AWK and Linux log analysis |
| **Team size** | 04 students (as recorded in the session notes) |
| **Team Lead** | Manogna |
| **Scribe** | Hemanth |
| **Screen-sharing / practical contributors** | Srivalli and Lokesh |

Prepared from the meeting transcript and the scribe’s session notes.

# 1. Session Overview

Day 02 focused on practical Linux command-line skills for analysing web-server log files. The team continued from the previous session, reviewed the earlier learning, divided the assigned questions, and worked through examples using **grep**, **cut**, **AWK**, pipes, sorting, and text-processing options.

The main practice file discussed during the session was **access.log**. The team used it to identify request patterns, extract selected fields, count responses, inspect IP addresses, and build summaries from log entries.

| Item | Details |
|---|---|
| Team | Kernel Panic – Team-02 |
| Recorded team size | 04 students |
| Session topic | Grep, Cut, AWK and related Linux commands |
| Working style | Screen sharing, question-wise task division, group discussion and troubleshooting |
| Scribe / documentation owner | Hemanth |

# 2. Team Roles & Contributions

The following roles are based on the scribe's notes and the meeting discussion. Manogna coordinated the team and assigned responsibilities. Hemanth was explicitly identified during the meeting as the scribe responsible for recording notes and collecting deliverables.

| Person | Role / contribution |
|---|---|
| Manogna | Team Lead. Assigned tasks and responsibilities, coordinated participation, monitored progress, and managed the overall team activity. She later had to leave because of a laptop/system issue, as recorded in the scribe's notes. |
| Hemanth | Scribe. Recorded the session, maintained the Word document, explained and discussed several grep questions, asked clarifying questions, and coordinated the collection of screenshots and command outputs. |
| Srivalli | Shared the screen and led practical demonstrations for the initial questions and the Cut section. Explained command behaviour and helped the team understand outputs. |
| Lokesh | Shared the screen during practical work and executed commands while the team discussed and verified the results. |
| Sai Kiran | Explained or guided parts of the assigned tasks and supported the group with command-related doubts, as noted by the scribe. |

# 3. Session Kick-off & Task Allocation

- The team began by revisiting concepts and activities from the previous class before moving to the Day 02 assignment.
- Manogna coordinated the team and distributed responsibilities so that work could continue if a member was unavailable.
- The first five questions were completed by Srivalli. The work was divided into sections so that members could participate in the practical exercises.

- Lokesh then shared the screen for the next set of questions. Hemanth explained the questions and discussed the commands, their purpose, and the expected output.
- The team used discussion and peer guidance whenever a command or output was unclear.

# 4. Task 01 – Searching Log Data with grep

The first major section focused on using **grep** to search for specific text and patterns in the access log. The team discussed how grep options change the output and how regular expressions can help match a group of status codes.

## Topics and activities discussed

- Searching for specific HTTP status codes, including 500 responses.
- Counting occurrences of POST requests.
- Displaying matching line numbers with the line-number option.
- Searching for user-agent text and request paths such as paths beginning with **/api/**.
- Counting error responses from the 4xx and 5xx status-code ranges.
- Using regular expressions and alternatives with the pipe symbol to match more than one status-code pattern.
- Understanding the difference between displaying matching lines and counting matches.

## Example command patterns discussed

```
grep -n "500" access.log
grep -c "POST" access.log
grep -E "[45][0-9]{2}" access.log
```

The exact option combination depends on the question. For example, a count of matching error responses can be produced by combining grep's extended-pattern and count options. The team also discussed why a line-number option is useful when the question asks where a match occurs.

**Learning outcome:** The team practised searching logs, counting matching requests, locating matching lines, and using regular expressions to identify status-code patterns.

# 5. Task 02 – Extracting Fields with cut

The next section introduced the **cut** command. The team explored how to extract selected fields or characters from each line of a file. This was a new topic for some members, so the group spent time understanding how delimiters and field numbers work.

## Key concepts

- **-d** specifies the delimiter used to split each line into fields.
- **-f** selects the field or fields to display.
- The field number depends on the structure of the log line and the delimiter being used.
- A space delimiter can separate parts of a log entry, while quotation marks may affect how request details are grouped.
- Pipes can pass the output of one command into another command for further extraction.

## Example discussed

```
cut -d " " -f1 access.log
```

The group used this pattern to extract the first field, which represented the IP address in the log format being practised. Members discussed how changing the field number changes the output, and why it is important to inspect an actual log line before deciding which field to select.

### Practical discussion

- Hemanth raised a doubt about how the Cut command works and how the delimiter determines the fields.
- Srivalli explained the delimiter and field-selection options while the team followed the command output.
- The group examined how request methods, timestamps, status codes, and user-agent details appear in different parts of a log entry.
- When the output did not match expectations, the team reviewed the log structure and adjusted the field selection.

**Learning outcome:** The team gained an initial understanding of delimiter-based extraction and learned that field positions must be interpreted according to the actual log format.

## 6. Task 03 – Processing Logs with AWK

The third section introduced **AWK** for field-based processing, conditional filtering, calculations, and formatted output. The team compared AWK with Cut and explored how AWK can select fields and perform operations on matching records.

### Concepts explored

- Using AWK field references such as **$1**, **$7**, **$9**, and **$10** to refer to fields in a log record.
- Using **print** to display selected fields or complete records.
- Applying conditions to select records with a particular status code, such as 500.
- Calculating totals by adding a numeric field such as the response-size/bytes field across records.
- Using associative arrays and counters to group or count requests by a field such as status code or IP address.
- Using **NR** to refer to the current record/line number and to report the total number of processed records.
- Using **printf** to produce a structured report containing selected values such as IP address, status code, and response size.

### Examples of AWK patterns discussed

```
awk '{ print $1, $7 }' access.log
awk '$9 == 500 { print $1, $7, $9 }' access.log
awk '{ total += $10 } END { print total }' access.log
awk '{ print NR, $0 } END { print NR }' access.log
```

These are representative patterns for the concepts discussed; the correct field numbers depend on the exact access-log format used in the exercise. The group checked field positions by examining sample log records and comparing the output.

### Group troubleshooting and learning

- The team initially found it difficult to identify the correct field numbers, especially for request paths, status codes, and response sizes.
- Members inspected individual log lines to understand how delimiters divide the record.
- The group corrected syntax issues, adjusted conditions, and tested the commands again.
- When a terminal became unresponsive, Hemanth suggested continuing in a new terminal.
- The team discussed how to calculate totals and how to print only the final result rather than every intermediate value.

**Learning outcome:** The team explored AWK's ability to extract fields, apply conditions, count records, calculate totals, and format output.

## 7. Collaborative Troubleshooting & Discussion

- The team discussed why a command was being used instead of simply copying it, with Hemanth helping explain several grep questions.
- Srivalli and other members guided the group through Cut syntax and field selection.
- Lokesh executed commands while the team observed the output and suggested corrections.
- The team used pipes to combine text-processing commands and discussed sorting and unique-value operations.
- When results were unexpected, members checked the input log format, delimiter positions, command options, and output.
- The final AWK exercises involved additional discussion around counters, conditions, totals, line numbers, and formatted output.

## 8. Documentation & Scribe Responsibilities

Hemanth’s scribe responsibility was an important part of the session. The meeting discussion confirmed that the scribe should record the work and submit the team’s deliverables. The team discussed collecting screenshots and command history so that the practical work could be reviewed later.

- Maintain the session notes in the shared Word document.
- Record the questions, commands used, explanations, and important troubleshooting steps.
- Ask members to capture screenshots showing a small group of related commands and their outputs.
- Organise screenshots with the contributor’s name and the relevant commands to avoid confusion.
- Collect command history and supporting evidence from team members.
- Share the completed document and deliverables with the team for review and submission.

The team also discussed sharing GitHub repository details and making the supporting material available for review by Harsha Anna.

# 9. Key Commands & Concepts – Quick Reference

| Command / concept | What it was used for in the session |
|---|---|
| grep | Search for matching text or patterns in access.log. |
| grep -n | Display line numbers for matching records. |
| grep -c | Count matching lines. |
| grep -E | Use extended regular expressions for pattern matching. |
| cut -d | Specify the delimiter used to split a line into fields. |
| cut -f | Select one or more fields from the input. |
| AWK $1, $7, $9, $10 | Refer to selected fields in each log record; field positions depend on the log format. |
| AWK print | Display selected fields or the full record. |
| AWK conditions | Filter records, for example by a status-code value. |
| AWK NR | Access the current record number and report the number of processed records. |
| sort / uniq | Sort values and work with unique entries, including IP-address lists. |
| Pipes (\|) | Send one command's output into another command. |
| printf | Format selected values into a readable report. |

# 10. Session Outcome

Day 02 provided hands-on practice with Linux text-processing tools using an access log. The team worked through the assigned questions in sections, shared the screen, explained command behaviour, compared outputs, and debugged issues together.

- **Grep:** Practised searching, counting, line-number display, and regular-expression matching.
- **Cut:** Learned delimiter-based field extraction and how to select fields from log records.
- **AWK:** Explored field references, conditions, counters, totals, record numbers, and formatted output.
- **Teamwork:** Practised collaborative problem-solving and shared responsibility for technical work and documentation.

The team worked through the questions together and wrapped up after the practical discussion. Some AWK exercises required additional experimentation, so the command outputs and notes should be retained for revision and verification.

# 11. Final Summary & Acknowledgement

The Day 02 session was focused on learning by doing. Instead of only studying command syntax, the team tested commands against access-log data, inspected outputs, discussed why particular options were required, and corrected mistakes together.

Special thanks to **Manogna** for coordinating the team and assigning responsibilities, **Srivalli** for leading practical demonstrations and explaining Cut, **Lokesh** for sharing the screen and running commands, and **Sai Kiran** for guiding the assigned topics. Thanks to all team members for contributing to the discussions.

**Hemanth – Scribe, Team-02 Kernel Panic:** Recorded the session, organised the learning points, and maintained the documentation and deliverables for the team.

---

Prepared by the team scribe using the Day 02 meeting transcript and the notes provided. Command examples are representative of the topics discussed; verify field numbers against the actual access-log format before reusing them.